



EcoCycle: E-Waste Management Portal

Complete Technical Architecture, Execution Guide & Top Interview Q&A

Tech Stack: Spring Boot 3.3.5, Java 21, MySQL 8.0, Thymeleaf 3

Developer: Saurabh Bhandari

Architecture: 3-Tier Layered MVC Pattern

Repository: github.com/bhandarisaurabh500/E-Waste-Management

1. Project Overview & Problem Statement

Domain & Value

The Problem: Rapid technological advancement has created an exponential increase in Electronic Waste (E-Waste). Discarded laptops, smartphones, batteries, and appliances contain hazardous elements like *Lead (Pb)*, *Mercury (Hg)*, *Cadmium (Cd)*, and *Brominated Flame Retardants*. Unregulated disposal leads to severe groundwater contamination, toxic air pollution, and massive loss of recoverable precious metals (Gold, Silver, Copper, Platinum).

The EcoCycle Solution: EcoCycle is a unified, automated digital portal designed to bring transparency and efficiency to e-waste disposal across three distinct stakeholders:

1. Individual Citizens

Schedule doorstep e-waste pickups with condition assessment, photo uploads, OTP verification, and instant receipts.

2. Corporate ITAD

Enterprise bulk decommissioning, EPR regulatory reporting, and certified NIST 800-88 data destruction.

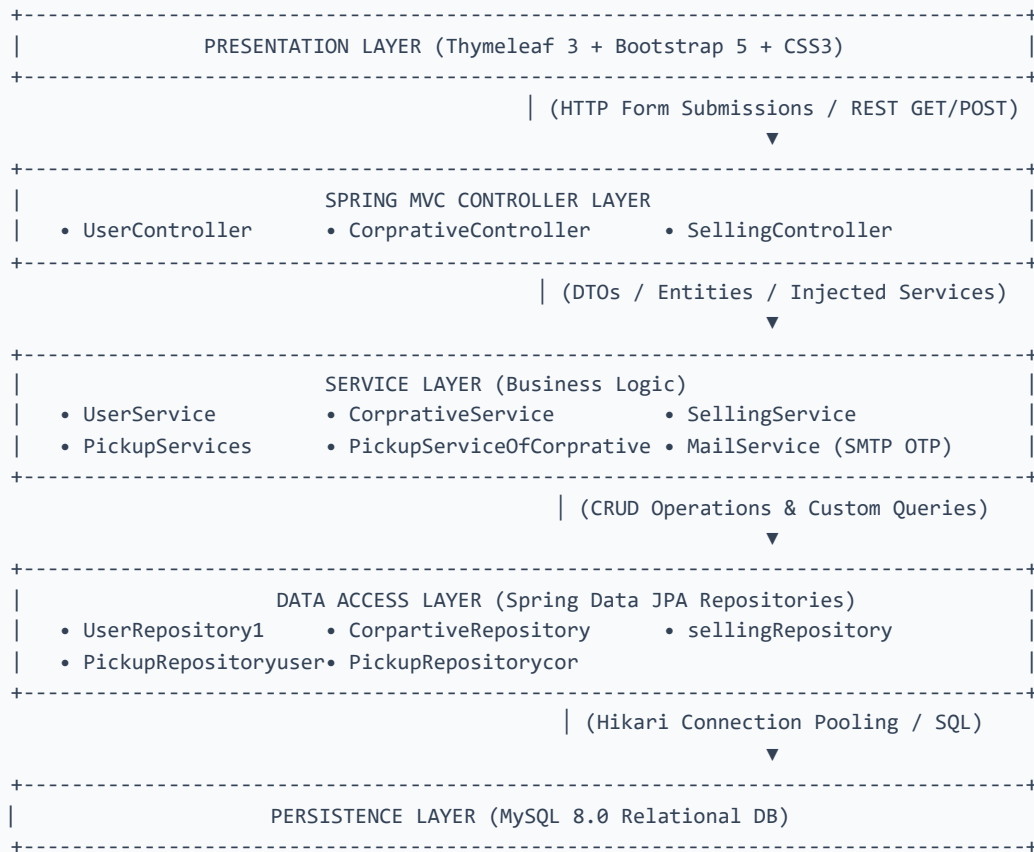
3. Certified Sellers

Circular economy secondary marketplace for refurbished Grade-A electronics with warranty & COD checkout.

2. Technical Architecture & Component Flow

Architecture

The application follows the industry-standard **Layered Spring MVC Pattern**, enforcing strict separation of concerns:



3. Database Schema & Entity Design

JPA / MySQL

Spring Data JPA automatically synchronizes the following entities with MySQL database tables via

`spring.jpa.hibernate.ddl-auto=update` :

Entity Class	Database Table	Key Fields & Attributes	Purpose / Usage
User.java	user	id (PK) , name , email , password , mobile , address	Individual citizen profiles and credentials.
Corprative.java	corprative	id (PK) , companyName , email , adress , contact , password	Corporate enterprise ITAD partners.
Pickup.java	pickup	id (PK) , productname , itemtype , damagepersentage , pickupdate , description , photoes	Citizen doorstep pickup requests.
PickupCorprater.java	pickup_corprater	id (PK) , productname , itemtype , damagepersentage , pickupdate , description , photoes	Corporate bulk IT asset pickup requests.
UserDtls.java	user_dtls	idsell (PK) , usernamesell , emailsell , contactsell , adresssell , passwordsell	Authorized refurbishers and scrap vendors.

4. Step-by-Step Execution Guide (How to Run)

Execution

Prerequisites: JDK 21 (LTS), Apache Maven 3.9+, MySQL Server running on port 3306 .

Step 1: Verify MySQL Database

```
mysql -u root -p
CREATE DATABASE IF NOT EXISTS e_wastemanagement;
```

Step 2: Configuration in `application.properties`

```
spring.datasource.url=jdbc:mysql://localhost:3306/e_wastemanagement?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
server.port=8080
```

Step 3: Launch via Terminal (PowerShell / Command Prompt)

```
$env:JAVA_HOME = "C:\Program Files\Java\jdk-21"
& "C:\Users\ASUS\Downloads\apache-maven-3.9.11-bin\apache-maven-3.9.11\bin\mvn.cmd" spring-boot:run
```

Step 4: Access in Web Browser

Navigate to `http://localhost:8080/` in Google Chrome / Edge.

5. Top 20 Technical Interview Questions & Answers

Interview Prep

Q1: Can you explain your project in 60 seconds (Elevator Pitch)?

💡 मराठी स्पष्टीकरण: प्रोजेक्टचा उद्देश, वापरलेले तंत्रज्ञान आणि 3 मुख्य रोलर्स थोडक्यात सांगणे.

"I developed **EcoCycle**, an enterprise-grade E-Waste Management & Circular Economy web platform built using **Spring Boot 3, Thymeleaf, Spring Data JPA, and MySQL**. It solves the critical environmental hazard of electronic waste by connecting three stakeholders: **Citizens** who can schedule doorstep e-waste pickups with OTP verification; **Corporates** who need bulk IT Asset Disposition (ITAD) and EPR compliance reports; and **Refurbishers** who sell certified second-life hardware on a secondary marketplace. I designed the MVC architecture, implemented email OTP authentication, designed the relational schema, and built a modern responsive UI."

Q2: Why did you choose Spring Boot over traditional Spring Framework?

💡 मराठी स्पष्टीकरण: Spring Boot मधील Auto-Configuration आणि Embedded Tomcat सर्वरचा फायदा.

"Spring Boot provides three major advantages:

1. **Auto-Configuration:** Automatically configures beans and database datasources based on classpath dependencies.
2. **Embedded Web Server:** Comes with embedded Tomcat (running on port 8080), eliminating the need for manual WAR deployments.
3. **Starter POMs:** `spring-boot-starter-web`, `spring-boot-starter-data-jpa`, and `spring-boot-starter-mail` simplify dependency management without XML boilerplate."

Q3: How does the OTP Email verification flow work internally?

💡 मराठी स्पष्टीकरण: युझरने ईमेल टाकल्यावर OTP कसा तयार होतो आणि JavaMailSender कसा काम करतो.

"When a user submits their registration form:

1. The controller calls `MailService.sendOtp(email)`.
2. `MailService` generates a random 6-digit number using `Random` or `SecureRandom`.
3. Using Spring's `JavaMailSender` configured with SMTP (`smtp.gmail.com`), a formatted HTML email is sent.
4. The generated OTP is stored in the `HttpSession`.
5. When the user submits the OTP, the controller validates the submitted value against the session OTP before persisting the record to MySQL via `userRepository.save(user)`."

Q4: What is the purpose of Thymeleaf Layout Fragments (`th:replace`)?

💡 मराठी स्पष्टीकरण: Header, Navbar आणि Footer प्रत्येक पेजवर पुन्हा न लिहिता base.html मधून reuse करणे.

"In traditional HTML, headers, navbars, and footers must be duplicated across every file. In EcoCycle, we created a single master layout `base.html`. Child templates (like `index.html`, `pickup.html`) use `th:replace=~{base::layout(~{::body})}`" to inject their page content into the base template. This follows the **DRY (Don't Repeat Yourself)** principle and ensures uniform styling across all 14 routes."

Q5: What are ITAD and EPR in E-Waste Management?

💡 **मराठी स्पष्टीकरण:** ITAD म्हणजे कंपनीचे कॉम्प्युटर सुरक्षित नष्ट करणे, आणि EPR म्हणजे सरकारी नियमांनुसार पुनर्वापर करणे.

"ITAD (IT Asset Disposition): The business practice of securely wiping, degaussing, and recycling obsolete corporate IT hardware in compliance with environmental and data security laws (such as NIST 800-88).

EPR (Extended Producer Responsibility): Environmental policy mandate where electronics producers and corporate consumers are legally obligated to ensure end-of-life recycling of the goods they produce or consume."

Q6: What is the difference between `@ModelAttribute` and `@RequestParam`?

💡 **मराठी स्पष्टीकरण:** फॉर्मचा पूर्ण Java Object बाइंड करणे विरुद्ध सिंगल केरी पॅरामीटर वाचणे.

"`@ModelAttribute` is used to bind an entire HTML form's input fields directly into a Java POJO/Entity (e.g., binding `Pickup` object in `/savepickup`).

`@RequestParam` is used to extract individual HTTP query parameters or simple form fields (e.g., reading `@RequestParam("otp") int otp`)."

Q7: How does Spring Data JPA simplify database operations?

💡 **मराठी स्पष्टीकरण:** JDBC किंवा SQL केरी न लिहिता JpaRepository द्वारे ऑटोमॅटिक CRUD ऑपरेशन्स मिळणे.

"By extending `JpaRepository<User, Integer>` , Spring Data JPA dynamically generates proxy implementations at runtime. It provides built-in CRUD methods like `save()` , `findById()` , `findAll()` , `delete()` , and query generation by method name (e.g., `findByEmail()`) without writing boilerplate SQL or JDBC Connection code."

Q8: How does Spring Boot handle database connection pooling?

💡 **मराठी स्पष्टीकरण:** HikariCP चा वापर करून डेटाबेस कनेक्शन जलद आणि कार्यक्षम ठेवणे.

"Spring Boot uses **HikariCP** as its default high-performance JDBC connection pool. Instead of opening and closing an expensive physical database connection for every incoming HTTP request, HikariCP maintains a pool of pre-established connections, significantly boosting throughput and reducing latency."

Q9: How do you handle file/image uploads for damaged e-waste devices?

💡 **मराठी स्पष्टीकरण:** युझरने अपलोड केलेला फोटो MultipartFile द्वारे कसा सेव्ह केला जातो.

"The HTML form uses `enctype="multipart/form-data"` . In the controller, Spring MVC receives the file as a `MultipartFile` parameter. The file can then be stored on the server filesystem in `static/img/uploads/` or encoded in Base64/cloud storage (AWS S3) and the filename is persisted in the `photoes` column of the `Pickup` database entity."

Q10: What is the function of `spring.jpa.hibernate.ddl-auto=update`?

💡 **मराठी स्पष्टीकरण:** Java मधील Entity क्लासेसनुसार डेटाबेसमधील टेबल्स आपोआप तयार किंवा अपडेट होणे.

"`ddl-auto=update`" instructs Hibernate to inspect the JPA Entity annotations (`@Entity` , `@Table` , `@Column`) at application startup and automatically update or create missing database tables and columns in MySQL without dropping existing data."

Q11: What HTTP methods are used in this project and when?

💡 **मराठी स्पष्टीकरण:** GET आणि POST मेथड्सचा वापर कुठे केला आहे.

GET: Used for fetching pages and resources (e.g., `GET /` for landing page, `GET /p` for pickup form, `GET /login/sell` for marketplace).

POST: Used for submitting sensitive data and mutating database state (e.g., `POST /saveuser` for registration, `POST /verifyOtp` , `POST /savepickup`)."

Q12: If this application has 100,000 users, how would you scale it?

💡 **मराठी स्पष्टीकरण:** भविष्यात युझर्स वाढल्यास सिस्टीम कशी मोठी (Scale) करता येईल.

"To scale EcoCycle:

1. **Stateless Auth:** Replace `HttpSession` with **JWT (JSON Web Tokens)** or distributed Redis session storage.
2. **Async Email Queue:** Use **RabbitMQ / Apache Kafka** with Spring `@Async` for non-blocking OTP email dispatch.
3. **Cloud Storage:** Move image uploads from local disk to **Amazon S3** bucket with CloudFront CDN.
4. **Database Optimization:** Implement MySQL Read-Replicas, database indexing on `email` and `pickupdate` , and Redis caching for the marketplace."